

To start Prometheus, type:

```
docker run -d --net=host \
> -v /root/prometheus.yml:/etc/
prometheus/prometheus.yml \
> --name prometheus-server \
> prom/prometheus
```

For starting the node exporter, the code snippet is:

```
docker run -d \
> -v "/proc:/host/proc" \
> -v "/sys:/host/sys" \
> -v "/:/rootfs" \
> --net="host" \
> --name=prometheus \
> quay.io/prometheus/node-
exporter:v0.13.0 \
> -collector.procfs /host/proc \>
-collector.sysfs /host/sys \
> -collector.filesystem.ignored-
mount-points "^(/sys|/proc|/dev|/host|etc)
($|/)"
```



Note: The paths shown here are purely local to a demo deployment. You may have to adjust the paths as per your own deployment.

1. Raw metrics can be viewed by `curl localhost:9100/metrics`. In the URL, it can be viewed in `/metrics` endpoint.
2. The metrics view can be found in `/targets` URL. For production, Grafana is the preferred tool.
3. The graph view can be found in `/graph` URL. Any metrics collected (viewable using `/metric` endpoint) can be viewed in `/graph`.

Sensu: Complementing Prometheus

Prometheus has lots of advantages because of its scalability, pluggability and resilience. It is tightly integrated with Kubernetes and has a thriving supportive community. However, it has a few disadvantages also. For example, the simplified constrained

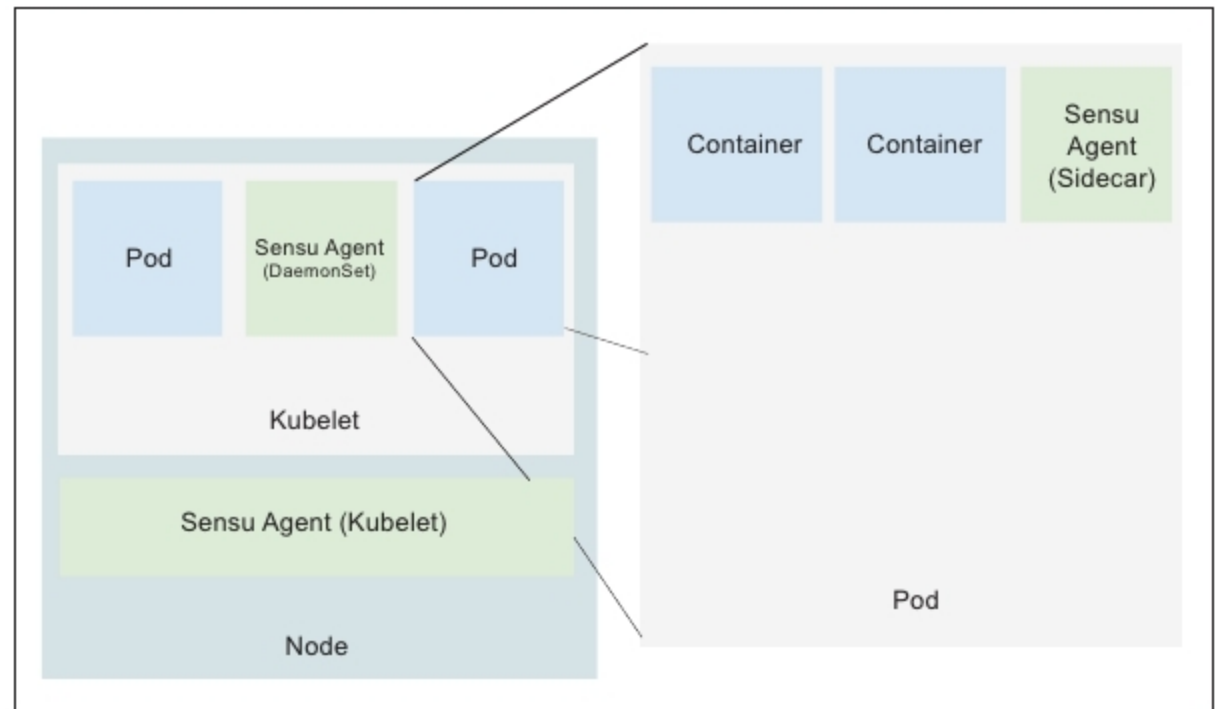


Figure 4: Sensu deployment (Reference: <https://blog.sensu.io/monitoring-kubernetes-docker-part-3-sensu-prometheus>)

data model loses various contexts of the measurements. It is tightly integrated with Kubernetes, so if the deployment is multi-generational, it may face challenges. The data transported is neither authenticated nor encrypted. Sensu complements Prometheus on this aspect aptly. Figure 4 shows how Sensu agents are typically deployed in a Kubernetes environment.

Some of the advantages Sensu provides to complement Prometheus are as follows:

1. Adds more context to Prometheus collected data to make it more meaningful.
2. Workflow automation of remediation actions based on alerts. So, alerts become actionable. This

is of paramount interest for self-healing purposes.

3. Secure transport: Prometheus does not encrypt or authenticate the metrics.

In this article, we discussed how microservices based architectural patterns are getting contextual in a DevOps environment with the ubiquitous usage of Kubernetes. Monitoring of such a sizable deployment becomes challenging because of its highly dynamic nature. So, we need tools like Prometheus to overcome some of the monitoring challenges. Sensu acts as a complementary tool to Prometheus, improving the latter's performance considerably. **END** 🐧

References

- [1] <https://dzone.com/articles/devops-tools-for-monitoring>
- [2] <https://blog.sensu.io/monitoring-kubernetes-part-1-the-challenges-data-sources>
- [3] <https://blog.sensu.io/monitoring-kubernetes-docker-part-2-prometheus>
- [4] <https://prometheus.io/docs/introduction/overview/>
- [5] <https://blog.sensu.io/monitoring-kubernetes-docker-part-3-sensu-prometheus>
- [6] <https://www.katacoda.com/courses/observability-analysis/prometheus>

By: Pradip Mukhopadhyay

The author has 19 years of experience across the stack – from low level systems programming to high level GUIs. He is a FOSS enthusiast and currently works for NetApp, Bengaluru.